

1. What is vectorization in NumPy?

Vectorization in NumPy means performing operations on entire arrays at once instead of using explicit Python loops. It allows you to write faster, cleaner, and more efficient code.

```
Without vectorization

import numpy as np

a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

c = []

for i in range(len(a)):
    c.append(a[i] + b[i])

print(c)

✓ 0.0s
[np.int64(11), np.int64(22), np.int64(33), np.int64(44)]

With Vectorization

import numpy as np

a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

c = a + b
print(c)

✓ 0.0s
[11 22 33 44]
```

2. What is the difference between reshape() and resize()?

Both the **numpy.reshape()** and **numpy.resize()** methods are used to change the size of a NumPy array. The difference between them is that the **reshape()** does not change the original array but only returns the changed array, whereas the **resize()** method returns nothing and directly changes the original array. That is, **reshape()** only changes the shape without altering data or the original array, whereas **resize()** changes both shape and size by modifying the original array.

```
reshape()
```

```
import numpy as np

a = np.array([1, 2, 3, 4, 5, 6])
b = a.reshape(2, 3)

print(b)
print(a)
```

✓ 0.0s

```
[[1 2 3]
 [4 5 6]]
[1 2 3 4 5 6]
```

```
resize()
```

```
a = np.array([1, 2, 3])
a.resize(2, 4)

print(a)
```

✓ 0.0s

```
[[1 2 3 0]
 [0 0 0 0]]
```

3. What are NumPy dimensions and axes?

NumPy dimensions

- Dimensions refer to the number of axes an array has.
- It is also called the rank of the array.
- In NumPy, dimensions are accessed using `ndim`.

NumPy axes

- An axis is a direction along which data is stored or operations are performed.
- Axes are numbered starting from 0.
- Axes are mainly used in aggregation functions like `sum()`, `mean()`, etc.
- Also, `axis=0` means Column wise and `axis=1` means row wise.

◇ *Relationship Between Dimensions and Axes*

- Number of axes = number of dimensions
- A 2D array has 2 axes (`axis=0`, `axis=1`)
- A 3D array has 3 axes (`axis=0`, `axis=1`, `axis=2`)

4. What is slicing in NumPy arrays?

Slicing in NumPy is used to extract a portion (sub-array) from an array by specifying a range of indices. It allows you to access multiple elements at once without using loops.

```
1D Array Slicing Example

import numpy as np

a = np.array([10, 20, 30, 40, 50])

print(a[1:4])    # [20 30 40]
print(a[:3])     # [10 20 30]
print(a[::-2])   # [10 30 50]
print(a[-3:])    # [30 40 50]

✓ 0.0s

[20 30 40]
[10 20 30]
[10 30 50]
[30 40 50]
```

5. How does NumPy help in mathematical computations for AI?

NumPy is a core library for numerical computing and plays a fundamental role in AI and Machine Learning by enabling fast, efficient, and scalable mathematical operations.

- ✓ **Efficient array and matrix operations**
 - Provides fast handling of multi-dimensional data using ndarray, essential for AI models.
- ✓ **Vectorization for high-speed computation**
 - Eliminates Python loops and performs operations on entire datasets at once, speeding up training.
- ✓ **Powerful linear algebra functions**
 - Supports matrix multiplication, inverses, eigenvalues, and solving equations used in ML algorithms.
- ✓ **Broadcasting mechanism**
 - Enables operations on arrays of different shapes without extra memory or loops.
- ✓ **Foundation for AI and ML libraries**
 - Acts as the base for libraries like TensorFlow, PyTorch, and Scikit-learn.

